EECS 348 — Assignment 4

GenAI Code Comparison: Sudoku Solver

Author: Prabesh Sapkota
Date: October 15, 2026

1. Objective and assignment requirements

The assignment asks for a recursive, object-oriented C++ program that reads puzzle1.txt through puzzle5.txt. Each puzzle is a 9×9 grid; digits 1–9 are given values and an underscore marks a blank cell. The program must print each file name and original puzzle, then print every solution or "No solution found." Since a puzzle may have multiple solutions, the solver must continue searching after finding one.

The two generated implementations are compared below. The final implementation selected for further work is the ChatGPT version, updated to use a Minimum Remaining Values (MRV) heuristic in addition to recursive backtracking.

2. GenAI programs and access method

GenAI 1 — ChatGPT: accessed through the ChatGPT interface. It generated the C++ implementation named *SudokuSolver* and assisted with the subsequent improvements.

GenAI 2 — Google Gemini: accessed through the Google Gemini website. It generated the alternative C++ implementation named *SudokuBoard*.

3. Prompt used

The assignment requires the same prompt to be used for both GenAI tools. The following is the standardized prompt documented for this comparison:

"Write a C++ program using object-oriented programming to solve five Sudoku puzzles stored in puzzle1.txt, puzzle2.txt, puzzle3.txt, puzzle4.txt, and puzzle5.txt. Each puzzle is a 9x9 grid, with underscores representing empty cells. Use recursive depth-first search with backtracking to solve the puzzles. The program must print the filename, the original puzzle, and every possible solution. If a puzzle has no solution, print 'No solution found.' Use a class to implement the Sudoku solver. Include comments and handle invalid input files."

Important: Ensure the prompt shown here is the exact prompt used in both tools; if not, replace it with the actual prompts or rerun both tools with the same prompt.

4. Overview of the generated programs

ChatGPT version: uses a *SudokuSolver* class and a 9×9 integer array, with 0 representing an empty cell. Its methods check whether a digit is safe, print the grid, load and validate a puzzle, check the original filled cells for conflicts, and recursively search for solutions. The final revised version also uses *findBestEmptyCell()* to select an empty cell with the fewest legal candidates.

Gemini version: uses a *SudokuBoard* class and a 9×9 character array, with underscore representing an empty cell. Its *isValid()* method checks row, column, and block constraints, while *solveHelper()* traverses cells in row-major order and uses recursive backtracking. Its file loader reads 81 cell characters but does not verify each extraction or validate conflicts among the original filled cells.

5. Comparison of correctness

ChatGPT implementation: The candidate-checking method enforces the three Sudoku rules: no duplicate digit in a row, column, or 3×3 block. The input reader rejects unexpected non-whitespace characters, accepts exactly 81 cell symbols, and reports a file-open error. It also checks the original givens for conflicts before search. The MRV selector chooses a cell with the fewest legal candidates; a cell with no candidates causes that search branch to fail, and recursive backtracking then tries alternatives.

Gemini implementation: The *isValid()* method checks row, column, and block rules for candidate digits. The recursive routine skips filled cells and tries digits 1–9 in empty cells. However, *loadFromFile()* does not check whether every extraction succeeded, so a short or malformed file may leave some grid cells uninitialized. It also does not detect conflicting original givens before search.

Assessment: By source inspection, the revised ChatGPT version has stronger input validation and clearer error handling. Neither implementation should be declared fully verified until it has been run against all five supplied puzzles and the output for puzzle1.txt has been compared with solution1.txt.

6. Execution-time analysis

Let k be the number of empty cells. In the worst case, a backtracking solver may try up to nine candidates at each empty cell, giving an exponential search bound of O(9^k) when the Sudoku board size is fixed. Each candidate check scans a row, a column, and a 3×3 block, which is constant work for a 9×9 puzzle.

The Gemini solver selects the next empty cell in row-major order. This is simple and has low selection overhead, but it may explore many branches when the chosen cell has many legal candidates. The revised ChatGPT solver scans empty cells and counts their legal candidates before choosing the cell with the fewest options. This MRV heuristic often reduces the search tree by exposing contradictions earlier, although it adds selection work at each recursive level and does not guarantee faster execution for every puzzle.

No numerical run-time comparison is reported because the same instructor puzzle files have not been benchmarked in the same environment. A fair timing comparison would compile both programs on the Cycle server, run the same puzzle inputs, and record elapsed time using the same method.

7. Space-complexity analysis

Both implementations store a fixed 9×9 board, so board storage is O(1) for the fixed-size assignment. Recursive calls can descend once per unresolved cell, giving O(k) stack space, where k is the number of blank cells. The ChatGPT loader temporarily stores up to 81 cell symbols in a vector; that is also O(1) for a fixed 9×9 board. Thus, both have O(k) auxiliary recursion space and constant-sized board storage for this assignment. The MRV improvement changes selection time, not the asymptotic recursion-space bound.

8. Maintainability analysis

ChatGPT: The responsibilities are separated into methods for validity checks, printing, loading, initial-board validation, cell selection, recursive solving, and overall solving. The revised code includes more detailed input checks and comments. The integer representation makes blank cells explicit as zero, while the input layer accepts underscores and zeros as blank markers.

Gemini: The class and helper methods are understandable and concise. The character grid closely matches the input file, and row-major traversal is straightforward. However, the file loader has less defensive checking, and the compact structure relies more heavily on valid input files.

Overall: ChatGPT is the preferred base because its input validation and initial-board checks make it more robust, and its revised MRV selector can reduce unnecessary search in many cases. Gemini remains a reasonable simple baseline, but its file handling should be strengthened before use on arbitrary input.

9. Selected program and justification

I selected the ChatGPT-generated SudokuSolver as the basis for the final program. The main reasons are its validation of the 81-cell input format, its handling of unexpected characters and file-open errors, and its check for conflicting original givens. The separate methods also make it easier to review the logic and extend the solver. The final version keeps recursive backtracking and adds MRV to improve the choice of the next cell.

10. Improvements made to the selected implementation

Correctness: Retained validation for file opening, allowed cell symbols, and exactly 81 cell symbols; retained the check for conflicts among original digits; and added MRV handling for an empty cell with zero legal candidates.

Execution time: Added the Minimum Remaining Values heuristic. At each search step, the program examines empty cells and selects one with the fewest legal candidate digits. This usually reduces branching on difficult puzzles, but it adds work to choose the next cell. Actual speed improvement must be measured on the provided puzzle files.

Space complexity: No asymptotic space reduction was introduced. The solver continues to use a fixed 9×9 board, a temporary vector of at most 81 cell symbols, and recursion proportional to the number of blank cells. MRV adds computation rather than a new large data structure.

Maintainability: Kept the solver in a single class with separate methods for input, validation, cell selection, printing, and recursive search. Added comments to explain the actions of code lines and blocks. The student should ensure the comments accurately reflect their understanding and the final program.

11. Compilation, debugging, and test plan

Compilation: The revised ChatGPT source was compiled locally with *g++ -std=c++17 -Wall -Wextra -pedantic* and completed without compiler diagnostics in that check. This does not replace the required Cycle-server or lab-machine check.

Required checks before submission:

1. Place puzzle1.txt through puzzle5.txt in the same working directory as the executable and run the program on the EECS Cycle server or lab machine.

2. Confirm that each output identifies the puzzle file and prints the original grid, then prints every solution or “No solution found.”

3. Compare the solver's result for puzzle1.txt with the instructor-provided solution1.txt.

4. Independently verify every printed solution: each row, column, and 3×3 block must contain digits 1–9 exactly once, and every original given must remain unchanged.

5. Test missing files, fewer than 81 cells, more than 81 cells, unexpected characters, conflicting givens, a valid puzzle with no solution, and a puzzle with multiple solutions if supplied or constructed for testing.

6. For a timing comparison, compile both implementations on the same Cycle server, run the same puzzle files, and record measurements using the same timing method. Do not include invented benchmark values.

Current status: Local compilation of the revised ChatGPT source is confirmed. The assignment puzzle files were not available for this report build, so puzzle-level correctness, runtime measurements, and Cycle-server compatibility remain to be verified.

12. Conclusion

Both GenAI programs implement recursive backtracking. ChatGPT provides stronger input validation, while Gemini uses a compact character grid and simple row-major traversal. The revised ChatGPT program was selected because of its validation and clearer separation of responsibilities. The added MRV heuristic may improve search efficiency, but its

benefit should be established by testing both programs against the same instructor files in the required environment.

Appendix A — Final revised ChatGPT-generated code

The listing below is taken from Assignment4.cpp.

```
/*
 * Program: EECS 348 Assignment 4
 * Description: Reads and solves five Sudoku puzzles using recursive backtracking.
 * Inputs: puzzle1.txt, puzzle2.txt, puzzle3.txt, puzzle4.txt, puzzle5.txt.
 * Output: Each original puzzle, every solution found, or an appropriate error message.
 * Collaborators: None.
 * Other sources: ChatGPT (OpenAI), used for code-generation assistance.
 * Author: Prabesh Sapkota
 * Creation Date: October 15, 2026
 * Note: Review, understand, compile, and test this code before submitting it.
 */
#include <iostream> // Provides console input and output.
#include <fstream> // Provides file-reading tools.
#include <string> // Provides the string type.
#include <vector> // Stores and validates the puzzle's 81 input cells.
#include <cctype> // Provides the isspace function.
#include <limits> // Provides numeric_limits for safe character handling.
using namespace std; // Allows standard-library names without the std:: prefix.
class SudokuSolver { // Defines a class that stores and solves one Sudoku puzzle.
private: // Begins the private section of the class.
    int board[9][9]; // Stores the Sudoku grid; zero represents an empty cell.
    int solutionCount; // Counts how many complete solutions have been found.
    bool isSafe(int row, int col, int digit) { // Checks whether digit can go in the specified cell.
        for (int j = 0; j < 9; j++) { // Checks all cells in the selected row.
            if (board[row][j] == digit) { // Detects the same digit in that row.
                return false; // Rejects a digit that already appears in the row.
            } // Ends the row-conflict condition.
        } // Ends the row scan.
        for (int i = 0; i < 9; i++) { // Checks all cells in the selected column.
            if (board[i][col] == digit) { // Detects the same digit in that column.
                return false; // Rejects a digit that already appears in the column.
            } // Ends the column-conflict condition.
        } // Ends the column scan.
        int startRow = (row / 3) * 3; // Finds the first row of the cell's 3-by-3 block.
        int startCol = (col / 3) * 3; // Finds the first column of the cell's 3-by-3 block.
        for (int i = startRow; i < startRow + 3; i++) { // Visits the three rows of the block.
            for (int j = startCol; j < startCol + 3; j++) { // Visits the three columns of the block.
                if (board[i][j] == digit) { // Detects the same digit inside the block.
                    return false; // Rejects a digit that already appears in the block.
                } // Ends the block-conflict condition.
            } // Ends the inner block scan.
        } // Ends the outer block scan.
        return true; // Confirms that the digit follows all Sudoku placement rules.
    } // Ends isSafe.
    void printBoard() const { // Prints the current Sudoku grid without modifying it.
        for (int i = 0; i < 9; i++) { // Visits each row of the board.
            if (i % 3 == 0) { // Checks whether a new group of three rows begins.
                cout << "+-------+-------+-------+" << endl; // Prints the horizontal grid border.
            } // Ends the row-group condition.
            for (int j = 0; j < 9; j++) { // Visits each cell in the current row.
                if (j % 3 == 0) { // Checks whether a new group of three columns begins.
                    cout << "| "; // Prints the left boundary of a 3-column group.
                } // Ends the column-group condition.
                if (board[i][j] == 0) { // Checks whether the current cell is empty.
                    cout << "_ "; // Displays an underscore for an empty cell.
                } else { // Handles a cell containing a digit.
                    cout << board[i][j] << " "; // Displays the digit stored in the cell.
                } // Ends the empty/non-empty condition.
            } // Ends the scan of the current row.
            cout << "|" << endl; // Prints the right boundary and starts a new output line.
        } // Ends the scan of all rows.
        cout << "+-------+-------+-------+" << endl; // Prints the bottom border of the grid.
    } // Ends printBoard.
    bool findBestEmptyCell(int& bestRow, int& bestCol) { // Finds an empty cell with the fewest legal
    candidates.
        int fewestCandidates = 10; // Starts above the maximum possible candidate count of nine.
        bestRow = -1; // Uses -1 to indicate that no empty cell has been selected yet.
        bestCol = -1; // Uses -1 to indicate that no empty cell has been selected yet.
        for (int row = 0; row < 9; row++) { // Searches each row of the board.
            for (int col = 0; col < 9; col++) { // Searches each column of the current row.
                if (board[row][col] != 0) { // Skips cells that already contain a digit.
                    continue; // Moves on to the next cell.
                } // Ends the non-empty-cell condition.
                int candidateCount = 0; // Counts legal digits for this empty cell.
                for (int digit = 1; digit <= 9; digit++) { // Tries each Sudoku digit as a candidate.
                    if (isSafe(row, col, digit)) { // Checks whether the candidate obeys Sudoku rules.
                        candidateCount++; // Counts this legal candidate.
                    } // Ends the candidate-validity condition.
                } // Ends the candidate-counting loop.
                if (candidateCount == 0) { // Detects an empty cell with no legal candidate.
                    bestRow = row; // Saves the contradictory cell's row.
                    bestCol = col; // Saves the contradictory cell's column.
                    return true; // Signals that an empty cell exists, but it has no possible value.
                } // Ends the no-candidate condition.
                if (candidateCount < fewestCandidates) { // Checks whether this cell has fewer candidates than
    the best so far.
                    fewestCandidates = candidateCount; // Records the new lowest candidate count.
                    bestRow = row; // Records the row of the best candidate cell.
                    bestCol = col; // Records the column of the best candidate cell.
                    if (fewestCandidates == 1) { // Checks whether this cell has only one possible digit.
                        return true; // A single-candidate cell cannot be improved upon.
                    } // Ends the single-candidate condition.
                } // Ends the best-cell update condition.
            } // Ends the scan of the current row.
        } // Ends the scan of the board.
        return bestRow != -1; // Returns true if an empty cell was found, otherwise the board is complete.
    } // Ends findBestEmptyCell.
    void solveRecursive() { // Searches recursively for all possible Sudoku solutions.
        int emptyRow = -1; // Holds the row of the selected empty cell.
        int emptyCol = -1; // Holds the column of the selected empty cell.
        if (!findBestEmptyCell(emptyRow, emptyCol)) { // Checks whether there are no empty cells left.
            solutionCount++; // Counts the completed valid board as a solution.
```

```
            cout << "\nSolution " << solutionCount << ":" << endl; // Prints the solution number.
            printBoard(); // Displays the completed Sudoku solution.
            return; // Backtracks to search for any additional solutions.
        } // Ends the completed-board condition.
        if (emptyRow == -1 || emptyCol == -1) { // Checks for an invalid selection state defensively.
            return; // Stops this branch if no valid cell coordinates were selected.
        } // Ends the defensive coordinate check.
        int candidateCount = 0; // Counts how many legal values the selected cell can take.
        for (int digit = 1; digit <= 9; digit++) { // Tests each digit for the selected cell.
            if (isSafe(emptyRow, emptyCol, digit)) { // Checks whether the digit is legal in this cell.
                candidateCount++; // Records that at least one valid candidate exists.
                board[emptyRow][emptyCol] = digit; // Places the candidate digit in the board.
                solveRecursive(); // Solves the remaining empty cells using recursion.
                board[emptyRow][emptyCol] = 0; // Removes the candidate to try another possibility.
            } // Ends the valid-candidate condition.
        } // Ends the digit-trial loop.
        if (candidateCount == 0) { // Identifies a branch where no digit can fit the selected cell.
            return; // Abandons this branch so the previous recursive call can backtrack.
        } // Ends the dead-end condition.
    } // Ends solveRecursive.
public: // Begins the public section of the class.
    SudokuSolver() : solutionCount(0) { // Initializes the solution counter when an object is created.
        for (int row = 0; row < 9; row++) { // Visits each row during initialization.
            for (int col = 0; col < 9; col++) { // Visits each column during initialization.
                board[row][col] = 0; // Marks every cell as empty.
            } // Ends the initialization of one row.
        } // Ends board initialization.
    } // Ends the constructor.
    bool loadPuzzle(const string& filename) { // Loads and validates the basic format of a puzzle file.
        ifstream inputFile(filename); // Opens the specified file for reading.
        if (!inputFile.is_open()) { // Checks whether the file opened successfully.
            cerr << "Error: Could not open " << filename << endl; // Reports a missing or inaccessible file.
            return false; // Stops loading when the file cannot be opened.
        } // Ends the file-open check.
        vector<char> cells; // Temporarily stores all 81 cell symbols before changing the board.
        char ch; // Stores each character read from the file.
        while (inputFile.get(ch)) { // Reads the file one character at a time.
            if ((ch >= '1' && ch <= '9') || ch == '_') { // Accepts digits one through nine and underscores.
                cells.push_back(ch); // Saves a valid Sudoku cell symbol.
            } else if (ch == '0') { // Allows zero as another way to mark an empty cell.
                cells.push_back('_'); // Normalizes zero to the underscore blank marker.
            } else if (!isspace(static_cast<unsigned char>(ch))) { // Rejects non-whitespace characters outside
    the accepted set.
                cerr << "Error: Invalid character in " << filename << endl; // Reports an unexpected character.
                return false; // Stops loading a malformed puzzle file.
            } // Ends the character-validation branches.
        } // Ends the file-reading loop.
        if (cells.size() != 81) { // Requires exactly 81 cell symbols for a 9-by-9 puzzle.
            cerr << "Error: " << filename << " must contain exactly 81 Sudoku cells." << endl; // Reports the
    incorrect cell count.
            return false; // Rejects a puzzle with too few or too many cells.
        } // Ends the cell-count check.
        for (int row = 0; row < 9; row++) { // Copies the validated symbols into each board row.
            for (int col = 0; col < 9; col++) { // Copies the symbols into each board column.
                char cell = cells[row * 9 + col]; // Retrieves the symbol at the current linear position.
                if (cell == '_') { // Checks whether the symbol represents a blank cell.
                    board[row][col] = 0; // Stores blank cells internally as zero.
                } else { // Handles a symbol containing a digit.
                    board[row][col] = cell - '0'; // Converts the digit character into its integer value.
                } // Ends the cell-conversion condition.
            } // Ends copying one row.
        } // Ends copying all 81 cells.
        return true; // Confirms that the file was read and formatted correctly.
    } // Ends loadPuzzle.
    bool isInitialBoardValid() { // Checks that the puzzle's original digits do not conflict.
        for (int row = 0; row < 9; row++) { // Examines each row of the starting puzzle.
            for (int col = 0; col < 9; col++) { // Examines each cell in the current row.
                if (board[row][col] == 0) { // Checks whether the current cell is blank.
                    continue; // Skips blank cells because they cannot conflict with a given digit.
                } // Ends the blank-cell condition.
                int digit = board[row][col]; // Saves the digit currently being checked.
                board[row][col] = 0; // Temporarily clears the cell so it does not conflict with itself.
                bool valid = isSafe(row, col, digit); // Checks the saved digit against its row, column, and
    block.
                board[row][col] = digit; // Restores the original digit after checking it.
                if (!valid) { // Checks whether another given digit conflicts with this one.
                    return false; // Rejects the starting puzzle if a conflict is found.
                } // Ends the conflict condition.
            } // Ends the scan of the current row.
        } // Ends the scan of the starting board.
        return true; // Confirms that all given digits are mutually consistent.
    } // Ends isInitialBoardValid.
    void solve(const string& filename) { // Prints a puzzle and searches for all its solutions.
        cout << "\n========================================" << endl; // Prints a separator before the puzzle
    heading.
        cout << "Puzzle file: " << filename << endl; // Identifies the file being processed.
        cout << "========================================" << endl; // Prints a separator after the puzzle
    heading.
        cout << "\nOriginal puzzle:" << endl; // Labels the original, unsolved puzzle.
        printBoard(); // Displays the board as it was loaded from the file.
        if (!isInitialBoardValid()) { // Checks for conflicting given digits before searching.
            cout << "\nNo solution found: the original puzzle has conflicting digits." << endl; // Explains why
    solving was stopped.
            return; // Skips backtracking for an invalid starting board.
        } // Ends the original-board validation condition.
        solutionCount = 0; // Resets the solution count before this search begins.
        solveRecursive(); // Starts recursive backtracking with the selected-cell heuristic.
        if (solutionCount == 0) { // Checks whether the search found no complete solution.
            cout << "\nNo solution found." << endl; // Reports that no valid solution exists.
        } else { // Handles a puzzle for which one or more solutions were found.
            cout << "\nTotal solutions found: " << solutionCount << endl; // Displays the number of solutions
    found.
        } // Ends the solution-result condition.
    } // Ends solve.
}; // Ends the SudokuSolver class definition.
int main() { // Defines the program's starting point.
    const string filenames[] = {"puzzle1.txt", "puzzle2.txt", "puzzle3.txt", "puzzle4.txt", "puzzle5.txt"}; //
    Lists the five required puzzle files.
    for (const string& filename : filenames) { // Processes each required filename in sequence.
```

```
        SudokuSolver solver; // Creates a fresh solver so puzzles do not share board state.
        if (solver.loadPuzzle(filename)) { // Loads the current file and continues only if its format is valid.
            solver.solve(filename); // Validates and solves the loaded puzzle.
        } // Ends the load-success condition.
    } // Ends processing of the five puzzle files.
    return 0; // Reports successful completion of the program.
} // Ends main.
```

Appendix B — Gemini-generated code

This is the alternative Gemini implementation retained for comparison.

```
/**
 * ============================================================================
 * Program Name: EECS 348 Assignment 4 - Sudoku Solver
 * Description: An object-oriented C++ program that solves 9x9 Sudoku puzzles
 *              read from text files using a recursive depth-first search with
 *              backtracking.
 * Inputs: puzzle1.txt, puzzle2.txt, puzzle3.txt, puzzle4.txt, puzzle5.txt
 * Outputs: The filename, initial puzzle grid, and all solved grid solutions
 *          (or "No solution found" if none exist).
 * Author: Prabesh Sapkota
 * Creation Date: October 15, 2026
 * Collaborators: AI Collaborator (Gemini) and Prabesh Sapkota
 * Other Sources: Assignment 4 Instructions / GenAI assistance for OOP structure
 * ============================================================================
 */

#include <iostream>
#include <fstream>
#include <vector>
#include <string>

using namespace std;

/**
 * Class representing a Sudoku Board and its solver logic using object-oriented principles.
 */
class SudokuBoard {
private:
    char grid[9][9]; // 9x9 grid storing characters ('1'-'9' or '_')

    /**
     * Helper function to check if placing a number at a given row and column is valid.
     * Checks the row, column, and the corresponding 3x3 block.
     */
    bool isValid(int row, int col, char num) {
        // Check row and column constraints
        for (int i = 0; i < 9; i++) {
            if (grid[row][i] == num) return false;
            if (grid[i][col] == num) return false;
        }

        // Check 3x3 block constraints
        int startRow = row - (row % 3);
        int startCol = col - (col % 3);
        for (int i = 0; i < 3; i++) {
            for (int j = 0; j < 3; j++) {
                if (grid[startRow + i][startCol + j] == num) return false;
            }
        }

        return true;
    }

    /**
     * Recursive backtracking function to find all solutions for the Sudoku puzzle.
     */
    bool solveHelper(int row, int col, bool& foundAny) {
        // Reached the end of the board, a solution is found
        if (row == 9) {
            foundAny = true;
            printGrid();
            cout << endl;
            return true;
        }

        // Move to the next row if column reaches 9
        if (col == 9) {
            return solveHelper(row + 1, 0, foundAny);
        }

        // Skip cells that are already filled
        if (grid[row][col] != '_') {
            return solveHelper(row, col + 1, foundAny);
        }

        bool localSolution = false;
        // Try digits '1' through '9'
        for (char num = '1'; num <= '9'; num++) {
            if (isValid(row, col, num)) {
                grid[row][col] = num;

                // Recurse for the next cell
                if (solveHelper(row, col + 1, foundAny)) {
                    localSolution = true;
                }

                grid[row][col] = '_'; // Backtrack
            }
        }
        return localSolution;
    }

public:
    SudokuBoard() {
        for (int i = 0; i < 9; i++) {
            for (int j = 0; j < 9; j++) {
                grid[i][j] = '_';
            }
        }
    }

    bool loadFromFile(const string& filename) {
```

```
        ifstream file(filename);
        if (!file.is_open()) {
            return false;
        }

        for (int i = 0; i < 9; i++) {
            for (int j = 0; j < 9; j++) {
                file >> grid[i][j];
            }
        }

        file.close();
        return true;
    }

    void printGrid() {
        for (int i = 0; i < 9; i++) {
            for (int j = 0; j < 9; j++) {
                cout << grid[i][j] << " ";
            }
            cout << endl;
        }
    }

    void solve(const string& filename) {
        cout << "Puzzle File: " << filename << endl;
        cout << "Original Puzzle:" << endl;
        printGrid();
        cout << endl;

        cout << "Solution(s):" << endl;
        bool foundAny = false;
        solveHelper(0, 0, foundAny);

        if (!foundAny) {
            cout << "No solution found" << endl;
        }
        cout << "----------------------------------------" << endl;
    }
};

int main() {
    string puzzleFiles[5] = {"puzzle1.txt", "puzzle2.txt", "puzzle3.txt", "puzzle4.txt", "puzzle5.txt"};

    for (int i = 0; i < 5; i++) {
        SudokuBoard board;
        if (board.loadFromFile(puzzleFiles[i])) {
            board.solve(puzzleFiles[i]);
        } else {
            cout << "Could not open file: " << puzzleFiles[i] << endl;
        }
    }

    return 0;
}
```